



**"RANKED 45
IN INDIA**

#University Category



**WORLD
UNIVERSITY
RANKINGS**

NO.1 PVT. UNIVERSITY IN
ACADEMIC REPUTATION IN INDIA



ACCREDITED GRADE 'A'
BY NAAC



PERFECT SCORE OF 150/150 AS A TESTAMENT
TO EXCEPTIONAL E-LEARNING METHODS

1



Applied Machine Learning

Unit 03 – Lecture 03: Gradient Descent

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

Stochastic, mini-batch and momentum

Topic focus: **how is the loss actually minimised?**

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Zhang, Lipton, Li and Smola, *Dive into Deep Learning*, Ch. 12 (Optimization Algorithms).; Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 7.; Kingma and Ba, "Adam: A Method for Stochastic Optimization," ICLR 2015.; Ruder, "An overview of gradient descent optimization algorithms," arXiv:1609.04747, 2016.

GD
○○○○○○

Step size
○○○○○

SGD
○○○○○

Mini-batch
○○○○○○○

Momentum
○○○○○○○○○○

Summary
○○

Quick Links

[Gradient descent](#)

[Learning rate](#)

[Stochastic](#)

[Mini-batch](#)

[Momentum](#)

[Summary](#)

GD
○○○○○

Step size
○○○○○

SGD
○○○○○

Mini-batch
○○○○○○○

Momentum
○○○○○○○○○

Summary
○○

Agenda

Gradient descent

The learning rate

Stochastic gradient descent

Mini-batch gradient descent

Momentum

Summary

How this lecture works

Every idea appears twice

1. **By hand** — small numbers, on paper, so you see the mechanism.
2. **In code** — the same thing, so you see the library does exactly that and nothing magic.

Commit before you look

Four slides ask you to predict an answer before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

Full derivations, more worked examples and 10 practice questions with solutions are in `notes.pdf`.

Where we are

Unit II gave us

What quantity do we minimise? MSE, MAE, Huber, cross-entropy, hinge.

Unit III (today)

How do we minimise it?

Next lecture

The adaptive optimisers: Adagrad, RMSprop, Adam.

Why the separation matters

The algorithms in this lecture do not care which loss you chose. One optimiser, every model. That is why it was worth answering the two questions separately.

Step 3, at last

1. Propose a family of candidate models, controlled by parameters.

Step 3, at last

1. Propose a family of candidate models, controlled by parameters.
2. Define **one number** saying how badly a candidate does.

(Unit II)

Step 3, at last

1. Propose a family of candidate models, controlled by parameters.
2. Define **one number** saying how badly a candidate does. (Unit II)
3. **Search the parameters to make that number small.** (today)

Step 3, at last

1. Propose a family of candidate models, controlled by parameters.
2. Define **one number** saying how badly a candidate does. (Unit II)
3. **Search the parameters to make that number small.** (today)

The whole of Unit III is one line

Step downhill. Repeat.

Step 3, at last

1. Propose a family of candidate models, controlled by parameters.
2. Define **one number** saying how badly a candidate does. (Unit II)
3. **Search the parameters to make that number small.** (today)

The whole of Unit III is one line

Step downhill. Repeat.

Everything interesting is in three questions: **how big** a step, computed from **how much data**, and with **how much memory** of the last step.

Why not just solve it exactly?

For linear regression with squared error we can: $w^* = (X^\top X)^{-1} X^\top y$.

Cost

$O(np^2 + p^3)$. At $p = 10^6$,
 $p^3 = 10^{18}$ operations.

Generality

Exists only because squared error is quadratic. Logistic loss: no closed form.

Memory

Needs all n rows at once. Streams and big data do not oblige.

The trade

Gradient descent swaps an exact answer computed *once* for an approximate answer improved *many times*. That is the only trade that scales.

The derivative already tells you which way to go

- $f'(w) > 0$: going right raises $f \Rightarrow$ move **left**.
- $f'(w) < 0$: going right lowers $f \Rightarrow$ move **right**.

The algorithm

$$w \leftarrow w - \eta f'(w)$$

Both say: move along $-f'(w)$.

η is the **learning rate**. With p parameters, replace f' by the gradient:
$$w \leftarrow w - \eta \nabla J(w).$$

Least squares: $\nabla J(w) = \frac{2}{n} X^\top (Xw - y)$. That is all the calculus this lecture needs.

By hand: five steps on $f(w) = (w - 3)^2$

$$f'(w) = 2(w - 3), \quad w_0 = 0, \quad \eta = 0.1, \quad \text{so } w \leftarrow 0.8w + 0.6.$$

k	w_k	$f(w_k)$	$f'(w_k)$
0	0.0000	9.0000	-6.0000
1	0.6000	5.7600	-4.8000
2	1.0800	3.6864	-3.8400
3	1.4640	2.3593	-3.0720
4	1.7712	1.5099	-2.4576

Look at the last column

−6.0, −4.8, −3.84, −3.072 — every step is exactly 0.8 times the one before. The algorithm brakes on its own as it approaches.

The same thing in code

```
def f(w): return (w - 3.0)**2
def df(w): return 2.0*(w - 3.0)
```

```
eta, w = 0.1, 0.0
for k in range(5):
    print("k=%d w=%.4f f(w)=%.4f f'(w)=%.4f" % (k, w, f(w), df(w)))
    w = w - eta*df(w)
```

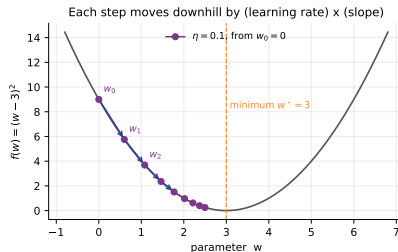
```
k=0 w=0.0000 f(w)=9.0000 f'(w)=-6.0000
k=1 w=0.6000 f(w)=5.7600 f'(w)=-4.8000
k=2 w=1.0800 f(w)=3.6864 f'(w)=-3.8400
k=3 w=1.4640 f(w)=2.3593 f'(w)=-3.0720
k=4 w=1.7712 f(w)=1.5099 f'(w)=-2.4576
```

Where the 0.8 comes from

Write $e_k = w_k - 3$ for the remaining error:

$$e_{k+1} = (1 - 2\eta) e_k \implies e_k = (1 - 2\eta)^k e_0.$$

With $\eta = 0.1$ the multiplier is 0.8, so $w_k = 3 - 3(0.8)^k$ — which prints exactly the table.



Convergence is geometric, not linear

Each step buys the same *percentage* of the remaining error, never the same amount. That is why every loss curve is a cliff followed by a long flat tail — and why the last decimal place costs the most.

Four regimes, from one formula

The error multiplier is $|1 - \eta f''|$. Everything follows.

Regime	Multiplier	Behaviour
η too small	just under 1	converges, but takes forever
η well chosen	near 0	converges fast, monotonically
η too large	in $(-1, 0)$	converges, overshooting and oscillating
η past the limit	> 1 in size	diverges

The stability threshold

$$0 < \eta < \frac{2}{L}, \quad L = \text{the largest curvature of } J. \quad \text{For } (w-3)^2: L=2, \text{ so } \eta < 1.$$

Predict first #1

Same problem, $w_0 = 0$, twenty iterations. $\eta < 1$ converges, $\eta > 1$ diverges.

What happens at η exactly 1.00? And at $\eta = 1.05$, five per cent past it?

Predict first #1

Same problem, $w_0 = 0$, twenty iterations. $\eta < 1$ converges, $\eta > 1$ diverges.

What happens at η exactly 1.00? And at $\eta = 1.05$, five per cent past it?

```
eta=0.10 |1-2*eta| = 0.80 w_20 = 2.9654
eta=0.50 |1-2*eta| = 0.00 w_20 = 3.0000
eta=0.90 |1-2*eta| = 0.80 w_20 = 2.9654
eta=1.00 |1-2*eta| = 1.00 w_20 = 0.0000
eta=1.05 |1-2*eta| = 1.10 w_20 = -17.1825
```

Predict first #1

Same problem, $w_0 = 0$, twenty iterations. $\eta < 1$ converges, $\eta > 1$ diverges.

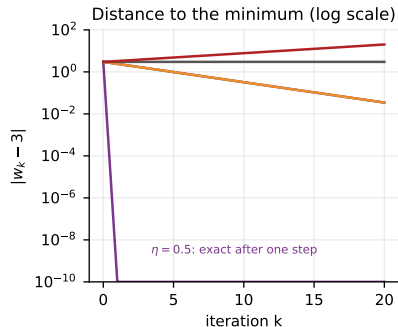
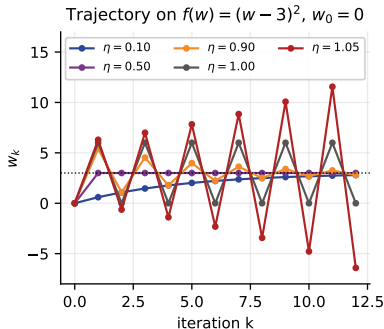
What happens at η exactly 1.00? And at $\eta = 1.05$, five per cent past it?

```
eta=0.10 |1-2*eta| = 0.80 w_20 = 2.9654
eta=0.50 |1-2*eta| = 0.00 w_20 = 3.0000
eta=0.90 |1-2*eta| = 0.80 w_20 = 2.9654
eta=1.00 |1-2*eta| = 1.00 w_20 = 0.0000
eta=1.05 |1-2*eta| = 1.10 w_20 = -17.1825
```

At $\eta = 1$ it bounces $0 \rightarrow 6 \rightarrow 0$ forever

Neutral: never converges, never diverges. Five per cent more and it is at -17.18 and accelerating.
There is no graceful degradation.

Two surprises, drawn



$\eta = 0.50$ is exact in one step

$\eta = 0.10$ and $\eta = 0.90$ tie

Diagnosing η from a loss curve

What you see

Still falling steeply when you stop

Falls fast, then flattens

Falls, then bounces without settling

Rises, or becomes nan

What to do

η too small: multiply by 3, or train longer

Healthy. Leave it alone

η slightly too large — or the SGD noise floor (later)

Past the threshold. Divide η by 10

A good η is a property of your data

L depends on how your features are scaled. A learning rate that works on standardised data can be hopeless on raw data — and we can measure exactly how hopeless.

Feature scaling is an optimisation decision

One feature recorded in metres instead of kilometres — multiply column 0 by 1000, change nothing else:

```
Xbad = load_diabetes().data.copy()
Xbad[:, 0] *= 1000.0
evb = np.linalg.eigvalsh(2.0/n * (Xbad.T @ Xbad))
```

```
condition number, standardised 470.1
condition number, one feature x1000 1.168e+08
stability limit 2/L, standardised 0.2485
stability limit 2/L, unscaled 4.420e-04
```

Feature scaling is an optimisation decision

One feature recorded in metres instead of kilometres — multiply column 0 by 1000, change nothing else:

```
Xbad = load_diabetes().data.copy()
Xbad[:, 0] *= 1000.0
evb = np.linalg.eigvalsh(2.0/n * (Xbad.T @ Xbad))
```

```
condition number, standardised 470.1
condition number, one feature x1000 1.168e+08
stability limit 2/L, standardised 0.2485
stability limit 2/L, unscaled 4.420e-04
```

The usable learning rate fell by a factor of 562

And iterations scale with the condition number, so the run got about 250,000× longer.
Standardise before you optimise.

Batch gradient descent, on real data

load_diabetes: 442 patients, 10 features, both standardised.

```
w = np.zeros(p)
for epoch in range(1, 6):
    w = w - 0.005*grad(X, y, w) # grad = 2/n * X.T @ (X@w - y)
    print("epoch %2d MSE %.4f" % (epoch, mse(X, y, w)))
```

```
epoch 1 MSE 0.9713
epoch 2 MSE 0.9447
epoch 3 MSE 0.9199
epoch 4 MSE 0.8969
epoch 5 MSE 0.8754
least-squares optimum MSE 0.4823
```

Epoch = one pass. **Update** = one application of the rule. Batch GD: **one update per epoch**, cost $O(np)$.

The structure we have been ignoring

$$J(w) = \frac{1}{n} \sum_{i=1}^n \ell_i(w) \quad \Rightarrow \quad \nabla J(w) = \frac{1}{n} \sum_{i=1}^n \nabla \ell_i(w)$$

The cost is an **average**. To estimate an average you do not need every term — you need a *sample*.

Pick one example i at random

$\mathbb{E}_i[\nabla \ell_i(w)] = \nabla J(w)$ — an **unbiased** estimate, at $1/n$ the cost.

$$w \leftarrow w - \eta \nabla \ell_i(w)$$

In one pass, SGD makes n updates where batch GD makes one — for the same total arithmetic.

By hand: three points, one epoch

Predict $\hat{y} = wx$ with $x = (1, 2, 3)$, $y = (2, 4, 7)$, $w_0 = 0$, $\eta = 0.05$. Gradient of one example: $2x_i(wx_i - y_i)$.

One batch step

$$\nabla J(0) = \frac{1}{3}(-4 - 16 - 42) = -20.67$$

$$w \leftarrow 0 + 0.05(20.67) = \mathbf{1.0333}$$

One epoch of SGD

$$x = 1: \quad 2(1)(0 - 2) = -4.00 \quad w = 0.2000$$

$$x = 2: \quad 2(2)(0.4 - 4) = -14.40 \quad w = 0.9200$$

$$x = 3: \quad 2(3)(2.76 - 7) = -25.44 \quad w = \mathbf{2.1920}$$

Exact optimum: $w^* = 31/14 = 2.2143$

Identical arithmetic — three per-example gradients either way. Batch reached 1.03; SGD reached 2.19, within one per cent. SGD's second gradient was evaluated at $w = 0.2$, not at $w = 0$.

The same thing in code

```
w = 0.0
g = np.mean(2*xs*(w*xs - ys))
print("one full-batch step: gradient %+.4f -> w = %.4f" % (g, w - eta*g)
)
w = 0.0 # now one epoch of SGD
for xi, yi in zip(xs, ys):
    gi = 2*xi*(w*xi - yi); w = w - eta*gi
    print(" SGD on x=%.0f: gradient %+.4f -> w = %.4f" % (xi, gi, w))
```

```
one full-batch step: gradient -20.6667 -> w = 1.0333
SGD on x=1: gradient -4.0000 -> w = 0.2000
SGD on x=2: gradient -14.4000 -> w = 0.9200
SGD on x=3: gradient -25.4400 -> w = 2.1920
```

Optimum 2.2143. In practice **shuffle every epoch**.

The price: SGD never quite arrives

Unbiased is not noiseless. Near the optimum the full gradient vanishes but the per-example ones do not, so SGD orbits in a cloud whose size grows with η .

```
eta = eta0/(1 + 0.002*t) if decay else eta0 # Robbins-Monro schedule
```

```
SGD (B=1, 30 epochs), constant step 0.02 MSE 0.5395  
SGD (B=1, 30 epochs), decayed step MSE 0.4834  
least-squares optimum MSE 0.4823
```

The price: SGD never quite arrives

Unbiased is not noiseless. Near the optimum the full gradient vanishes but the per-example ones do not, so SGD orbits in a cloud whose size grows with η .

```
eta = eta0/(1 + 0.002*t) if decay else eta0 # Robbins-Monro schedule
```

```
SGD (B=1, 30 epochs), constant step 0.02 MSE 0.5395  
SGD (B=1, 30 epochs), decayed step MSE 0.4834  
least-squares optimum MSE 0.4823
```

$$\sum \eta_t = \infty \text{ and } \sum \eta_t^2 < \infty$$

The first says do not stop before arriving; the second says damp the noise. $\eta_t = \eta_0/(1 + ct)$ does both — and closed almost the whole remaining gap here.

Nothing forces B to be 1 or n

$$w \leftarrow w - \eta \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \ell_i(w), \quad \mathcal{B} \text{ random, } |\mathcal{B}| = B$$

	B	updates/epoch	noise	cost/update
Batch	n	1	none	$O(np)$
Mini-batch	32–512	n/B	moderate	$O(Bp)$
Stochastic	1	n	large	$O(p)$

Batch and SGD are the two endpoints. The answer is always in between — and this section is why.

Predict first #2

Thirty epochs, same η , same total arithmetic per epoch. SGD reaches MSE **0.5268** after *one* epoch.

How many epochs does full-batch gradient descent need to reach the same MSE?

Predict first #2

Thirty epochs, same η , same total arithmetic per epoch. SGD reaches MSE **0.5268** after *one* epoch.

How many epochs does full-batch gradient descent need to reach the same MSE?

```
updates/epoch after 1 epoch after 30 epochs
batch B=442 1 0.9713 0.6219
mini B= 32 14 0.7393 0.4860
SGD B= 1 442 0.5268 0.4859
Full-batch GD needs 80 epochs to reach the same MSE.
```

Predict first #2

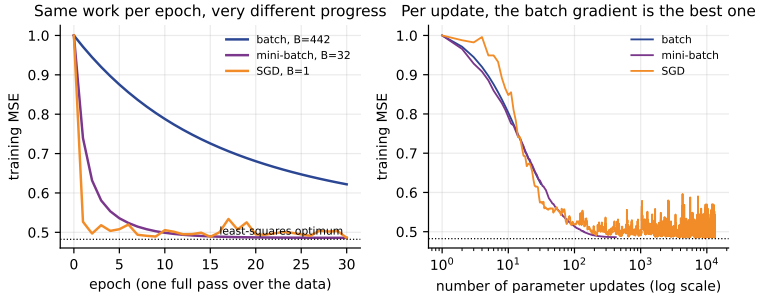
Thirty epochs, same η , same total arithmetic per epoch. SGD reaches MSE **0.5268** after *one* epoch.

How many epochs does full-batch gradient descent need to reach the same MSE?

```
updates/epoch after 1 epoch after 30 epochs
batch B=442 1 0.9713 0.6219
mini B= 32 14 0.7393 0.4860
SGD B= 1 442 0.5268 0.4859
Full-batch GD needs 80 epochs to reach the same MSE.
```

Eighty passes against one (optimum 0.4823). All of it comes from updating 442 times per pass instead of once.

Two panels that contradict each other — both true



Per **update**, the batch gradient is the best one (right). Per unit of **work**, it is by far the worst (left). We are judged on the second, because work is what you pay for.

How noisy is one example, really?

```
g_full = grad(X, y, w0)
for B in [1, 2, 4, 8, 16, 32, 64, 128, 256]:
    ... # RMS of ||grad on a random batch of B - g_full||, 3000 draws
```

```
|| full gradient || 2.4157
B= 1 RMS ||g_B -g|| = 6.3234
B= 2 RMS ||g_B -g|| = 4.4099
B= 4 RMS ||g_B -g|| = 3.1309
B= 8 RMS ||g_B -g|| = 2.1805
B= 16 RMS ||g_B -g|| = 1.5199
B= 32 RMS ||g_B -g|| = 1.0641
```

How noisy is one example, really?

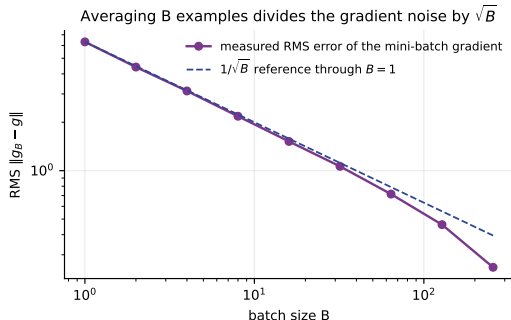
```
g_full = grad(X, y, w0)
for B in [1, 2, 4, 8, 16, 32, 64, 128, 256]:
    ... # RMS of ||grad on a random batch of B - g_full||, 3000 draws
```

```
|| full gradient || 2.4157
B= 1 RMS ||g_B -g|| = 6.3234
B= 2 RMS ||g_B -g|| = 4.4099
B= 4 RMS ||g_B -g|| = 3.1309
B= 8 RMS ||g_B -g|| = 2.1805
B= 16 RMS ||g_B -g|| = 1.5199
B= 32 RMS ||g_B -g|| = 1.0641
```

Read the first two lines together

The true gradient has length 2.42; a single-example gradient is typically wrong by 6.32. **A single SGD step often points uphill.** It works only because the errors cancel.

The $1/\sqrt{B}$ law



Each doubling of B

multiplies the error by $1/\sqrt{2} = 0.707$:
 $6.32 \rightarrow 4.41 \rightarrow 3.13 \rightarrow 2.18$

The decisive trade

Noise falls as $\sqrt{B}$; cost grows as B .
Doubling B buys 41% less noise for 100% more work.

$B=1 \rightarrow B=32$: $32\times$ the work, $5.9\times$ less noise ($\sqrt{32} = 5.66$).

Predict first #3

$B = 32$ does $32\times$ the arithmetic per update that $B = 1$ does. Both reach essentially the same MSE (0.4860 against 0.4859 after 30 epochs).

Per epoch — identical total arithmetic — which is faster in wall-clock time, and by how much?

Predict first #3

$B = 32$ does $32\times$ the arithmetic per update that $B = 1$ does. Both reach essentially the same MSE (0.4860 against 0.4859 after 30 epochs).

Per epoch — identical total arithmetic — which is faster in wall-clock time, and by how much?

```
B= 1 442 updates/epoch 1.224 ms per epoch  
B= 32 14 updates/epoch 0.040 ms per epoch  
B= 442 1 updates/epoch 0.005 ms per epoch
```


Predict first #3

$B = 32$ does $32\times$ the arithmetic per update that $B = 1$ does. Both reach essentially the same MSE (0.4860 against 0.4859 after 30 epochs).

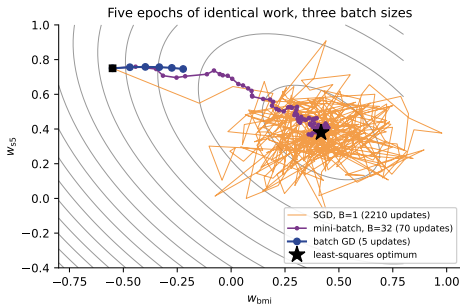
Per epoch — identical total arithmetic — which is faster in wall-clock time, and by how much?

```
B= 1 442 updates/epoch 1.224 ms per epoch  
B= 32 14 updates/epoch 0.040 ms per epoch  
B= 442 1 updates/epoch 0.005 ms per epoch
```

$B = 32$ is 31 times faster than $B = 1$

Same arithmetic, one vectorised matrix operation instead of 32 tiny ones. On a GPU the effect is far larger — thousands of cores sit idle processing one example at a time.

Why mini-batch wins



Five epochs of identical work. Batch GD barely leaves the start; SGD arrives fast, then rattles around the optimum.

Better than both ends, not a compromise

- Keeps almost all of SGD's update frequency — 14 per epoch was enough here.
- Recovers vectorised arithmetic: $31\times$ on this laptop, far more on a GPU.
- Cuts gradient noise by $\sqrt{B}$ into the bargain.

Terminology in the wild

Almost everything called “SGD” is mini-batch GD. **Start at $B = 32$, powers of two, tune η first.**

The ravine

Measured on our data

Largest curvature 8.05, smallest 0.017.

Condition number $\kappa = 470$.

A long, narrow valley: steep walls, gently sloping floor.

The bind

$\eta < 2/L$ is set by the **steep walls**.

The progress you want is along the **floor**, where the curvature is 470 times smaller.

So the iterates bounce from wall to wall and creep along the valley.

Gradient descent is limited by the direction you do not want to travel in.

The heavy ball

Stop thinking of a hiker taking independent steps. Think of a ball rolling.

Along the floor	slope points the same way $\Rightarrow$ velocity <i>builds</i>
Across the valley	slope reverses each step $\Rightarrow$ contributions <i>cancel</i>

Polyak's heavy ball — the form PyTorch and D2L use

$$v \leftarrow \beta v + \nabla J(w) \quad w \leftarrow w - \eta v$$

$\beta = 0$ recovers plain gradient descent exactly.

Momentum is an exponentially weighted average

Unroll it from $v_0 = 0$:

$$v_t = \beta v_{t-1} + g_t = g_t + \beta g_{t-1} + \beta^2 g_{t-2} + \cdots = \sum_{j \geq 0} \beta^j g_{t-j}$$

$$\text{Total weight} = \sum_{j \geq 0} \beta^j = \frac{1}{1 - \beta}$$

The number to memorise

$$\eta_{\text{eff}} \approx \frac{\eta}{1 - \beta} \quad \beta = 0.9 \Rightarrow 10\eta; \quad \beta = 0.99 \Rightarrow 100\eta$$

A $10\times$ speed-up in consistent directions, without raising η in the inconsistent ones.

By hand: four steps of momentum

$$f(w) = (w - 3)^2, \quad \eta = 0.1, \quad \beta = 0.9, \quad w_0 = 0, \quad v_0 = 0.$$

k	g_k	$v_k = 0.9 v_{k-1} + g_k$	$w_k = w_{k-1} - 0.1 v_k$
1	-6.0000	-6.0000	$0 + 0.6000 = 0.6000$
2	-4.8000	-10.2000	$0.6 + 1.0200 = 1.6200$
3	-2.7600	-11.9400	$1.62 + 1.1940 = 2.8140$
4	-0.3720	-11.1180	$2.814 + 1.1118 = \mathbf{3.9258}$

Plain GD was at 1.7712 after four steps

Momentum is at 3.9258 — already **past** the minimum at 3. Row 4 shows how: the gradient is nearly zero (-0.372) but the velocity is still -11.12 . The ball arrives travelling fast.

Predict first #4

Same one-dimensional bowl, $\eta = 0.1$, starting at $w_0 = 0$ with the minimum at $w = 3$. Plain GD reaches $|w - 3| < 10^{-3}$ in 36 steps.

With $\beta = 0.9$: faster or slower? And how far past 3 does it go?

Predict first #4

Same one-dimensional bowl, $\eta = 0.1$, starting at $w_0 = 0$ with the minimum at $w = 3$. Plain GD reaches $|w - 3| < 10^{-3}$ in 36 steps.

With $\beta = 0.9$: faster or slower? And how far past 3 does it go?

```
beta=0.0 first k with |w-3|<1e-3: 36 highest w reached: 3.0000  
beta=0.5 first k with |w-3|<1e-3: 20 highest w reached: 3.2092  
beta=0.9 first k with |w-3|<1e-3: 92 highest w reached: 5.1269
```


Predict first #4

Same one-dimensional bowl, $\eta = 0.1$, starting at $w_0 = 0$ with the minimum at $w = 3$. Plain GD reaches $|w - 3| < 10^{-3}$ in 36 steps.

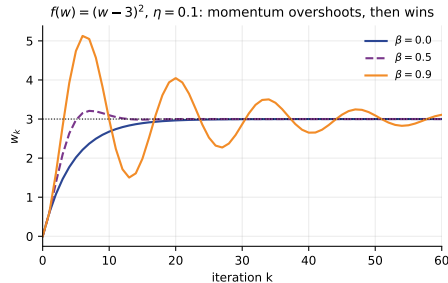
With $\beta = 0.9$: faster or slower? And how far past 3 does it go?

```
beta=0.0 first k with |w-3|<1e-3: 36 highest w reached: 3.0000  
beta=0.5 first k with |w-3|<1e-3: 20 highest w reached: 3.2092  
beta=0.9 first k with |w-3|<1e-3: 92 highest w reached: 5.1269
```

$\beta = 0.9$ is two and a half times *slower*

And it sails out to 5.13 — almost as far past the minimum as it started short of it. A 1-D quadratic has $\kappa = 1$: no ravine, so nothing for momentum to fix, and only the overshoot to pay for.

Overshoot, drawn



Momentum is not a free improvement

It is a targeted remedy for **ill-conditioned** problems — which describes almost every real model, but not this one.

Now put it in a ravine

$f(w) = 0.01w_1^2 + 2w_2^2$, so $\kappa = 4/0.02 = 200$. Plain GD's ceiling is $2/L = 0.5$; we use $\eta = 0.475$ throughout.

```
plain-GD ceiling 2/L = 0.500
momentum ceiling 2(1+beta)/L = 0.950 (beta=0.9)
eta=0.475 beta=0.0 iterations to ||w||<1e-3: 954
eta=0.475 beta=0.5 iterations to ||w||<1e-3: 467
eta=0.475 beta=0.9 iterations to ||w||<1e-3: 135
eta=0.900 beta=0.0 iterations to ||w||<1e-3: diverged at k=20
eta=0.900 beta=0.9 iterations to ||w||<1e-3: 161
```

Now put it in a ravine

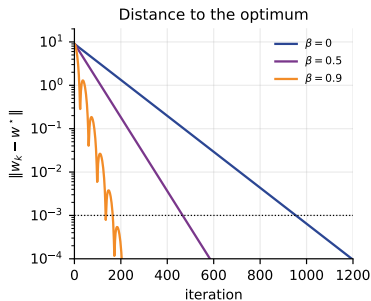
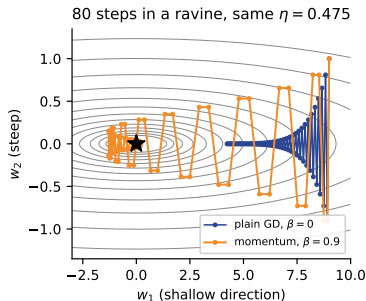
$f(w) = 0.01w_1^2 + 2w_2^2$, so $\kappa = 4/0.02 = 200$. Plain GD's ceiling is $2/L = 0.5$; we use $\eta = 0.475$ throughout.

```
plain-GD ceiling  $2/L = 0.500$   
momentum ceiling  $2(1+\text{beta})/L = 0.950$  (beta=0.9)  
eta=0.475 beta=0.0 iterations to  $\|w\| < 1e-3$ : 954  
eta=0.475 beta=0.5 iterations to  $\|w\| < 1e-3$ : 467  
eta=0.475 beta=0.9 iterations to  $\|w\| < 1e-3$ : 135  
eta=0.900 beta=0.0 iterations to  $\|w\| < 1e-3$ : diverged at k=20  
eta=0.900 beta=0.9 iterations to  $\|w\| < 1e-3$ : 161
```

954 $\rightarrow$ 135: a $7.1\times$ speed-up from one extra line

And the last two rows: at $\eta = 0.9$ plain GD explodes, while momentum converges — because momentum's ceiling is $2(1 + \beta)/L$, not $2/L$.

The ravine, drawn — and it is not what you expect



Momentum's path is **messier**, not smoother. What it is not, is stuck. It trades cross-valley tidiness for along-valley speed — in a ravine, always a trade you want.

On the real dataset, and with mini-batches

```
beta=0.0 eta=0.2361 full-batch iterations to the optimum: 1605  
beta=0.9 eta=0.2361 full-batch iterations to the optimum: 128
```

$\kappa = 470$ on the diabetes problem, so $12.5\times$ — more than the $7.1\times$ at $\kappa = 200$. **Worse conditioning, more benefit.**

```
mini-batch B=32, eta=0.005, 5 epochs  
  beta=0.00 MSE 0.5358  
  beta=0.90 MSE 0.4870  
  beta=0.99 MSE 0.5674  
least-squares optimum MSE 0.4823
```

On the real dataset, and with mini-batches

```
beta=0.0 eta=0.2361 full-batch iterations to the optimum: 1605  
beta=0.9 eta=0.2361 full-batch iterations to the optimum: 128
```

$\kappa = 470$ on the diabetes problem, so $12.5\times$ — more than the $7.1\times$ at $\kappa = 200$. **Worse conditioning, more benefit.**

```
mini-batch B=32, eta=0.005, 5 epochs  
  beta=0.00 MSE 0.5358  
  beta=0.90 MSE 0.4870  
  beta=0.99 MSE 0.5674  
least-squares optimum MSE 0.4823
```

$\beta = 0.99$ is *worse than no momentum*: the effective step is 100η and the run overshoots.

Two things that will cost you an afternoon

When you raise β , lower η

Effective step is $\eta/(1 - \beta)$. Moving β from 0.9 to 0.99 multiplies your learning rate by **ten** without you touching it. Rescale by $(1 - \beta_{\text{new}})/(1 - \beta_{\text{old}})$.

Two conventions, differing by a factor of $1 - \beta$

PyTorch, D2L, these slides	$v \leftarrow \beta v + g$
Andrew Ng, the Adam paper	$v \leftarrow \beta v + (1 - \beta)g$

Both correct; identical if η is scaled by $1/(1 - \beta)$. Copy η across without rescaling and your learning rate silently changes tenfold.

Summary

1. $w \leftarrow w - \eta \nabla J(w)$. On a quadratic the error is multiplied by a fixed factor each step: convergence is **geometric**.
2. Hard ceiling $\eta < 2/L$. At $\eta = 1.00$ our run bounced forever; at 1.05 it hit -17.18 in twenty steps. No graceful degradation.
3. $\eta = 0.1$ and $\eta = 0.9$ converged at *identical* rates. Bigger is not faster.
4. The cost is an average, so one random example gives an **unbiased** gradient at $1/n$ the price. That is SGD.
5. It is very noisy: true gradient length 2.42, single-example error 6.32. Steps often point uphill; the errors cancel.
6. Per epoch, small batches win: SGD reached 0.5268 in one epoch; batch GD needed **80**.
7. Noise falls as $1/\sqrt{B}$, cost grows as B . $B=32$ measured $5.9\times$ less noise than $B=1$ for $32\times$ the arithmetic.
8. **Mini-batch wins because it vectorises**: same MSE as $B=1$, $31\times$ faster per epoch.
9. $v \leftarrow \beta v + \nabla J$: an exponentially weighted average, effective step $\eta/(1-\beta)$.
10. Momentum **overshoots** — to 5.13 past a minimum at 3 — and on a well-conditioned problem it is slower.
11. In a ravine it is transformative: $954 \rightarrow 135$ at $\kappa = 200$, $1605 \rightarrow 128$ at $\kappa = 470$. It also raises the ceiling to $2(1+\beta)/L$.
12. Standardising cut κ from 1.2×10^8 to 470. Conditioning is something you control.

Before the next lecture

Read

notes.pdf — the full derivations, the Robbins–Monro conditions, and **10 practice questions with worked solutions**.

Do

Questions 1, 6 and 10. Question 6 is momentum by hand; question 10 ties scaling, conditioning and the learning rate together.

Next: adaptive optimisers

Momentum gives every coordinate the same η and the same β .

L4 asks what happens when each coordinate gets its own step size: Adagrad, RMSprop, Adadelata, Adam.

T1 follows this lecture

Units I–II, 40 minutes.

Materials: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Zhang, Lipton, Li and Smola, *Dive into Deep Learning*, Ch. 12 (Optimization Algorithms); Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 7; Kingma and Ba, "Adam: A Method for Stochastic Optimization," ICLR 2015; Ruder, "An overview of gradient descent optimization algorithms," arXiv:1609.04747, 2016.